# AI-Native Backend Notes

AI-Native Backend Notes

Generated: 15/03/2026

Daniel Rosehill

# Contents


1 Traditional CMS vs. MCP-Driven Blog: Development Trajectories Compared . . . 3
1.1 Overview . . . 3
1.2 Side-by-Side Development Timeline . . . 3
1.3 The Shared Foundation . . . 4
1.4 The Point of Divergence . . . 4
1.5 What Becomes Redundant in the MCP-Driven Approach . . . 5
1.6 What Increases in Importance . . . 6
1.7 Development Effort Comparison . . . 8
1.8 Tradeoffs and Practical Considerations . . . 8
1.9 Summary . . . 9
2 Sequential Order of Backend Development Operations with an Admin MCP Server . . . 10
2.1 The Sequential Steps at a Glance . . . 10
2.2 Step 1: Domain Modelling and Schema Definition . . . 10
2.3 Step 2: Database Provisioning and Migration . . . 11
2.4 Step 3: Shared Data-Access Layer . . . 12
2.5 Step 4: Public API Creation . . . 13
2.6 Step 5: Admin MCP Server Creation . . . 14
2.7 Step 6: Security, Authorization, and Audit Logging . . . 18
2.8 Step 7: Deployment and Runtime Configuration . . . 19
2.9 Summary: The Dependency Chain . . . 20
2.10 Why Replace the Admin Panel with an MCP Server? . . . 20
2.11 Sources . . . 21
3 Dual MCP Server Architecture: Public and Admin Access . . . 22
3.1 The Scenario: CityTransit – A Public Transit Data Platform . . . 22
3.2 The Two MCP Servers . . . 22
3.3 Tool Definitions . . . 23
3.4 Development Approach . . . 25
3.5 Security Considerations . . . 26
3.6 Deployment and Operational Concerns . . . 28
3.7 Summary . . . 29
4 Tooling for Unified API and MCP Tool Development with Visibility . . . 31
4.1 The Problem . . . 31
4.2 Tier 1: Framework-Level Libraries (Define Once, Expose Both) . . . 31
4.3 Tier 2: Code Generation Platforms (OpenAPI as Single Source of Truth) . . . 32
4.4 Tier 3: Gateway and Registry Infrastructure (Centralized Visibility) . . . 33
4.5 Emerging Patterns . . . 34
4.6 Practical Recommendations . . . 34
4.7 Summary Table . . . 35

*Question:*

Consider two development projects, both building a blog: (1) a blog with a bespoke CMS backend (traditional approach), and (2) a blog where the backend is entirely MCP-driven, with a conversational AI agent as the intended interface. Chart the development trajectories side by side, identify where they diverge, and which steps become redundant or increase in importance.

# 1 Traditional CMS vs. MCP-Driven Blog: Development Trajectories Compared

## 1.1 Overview

Building a blog sounds simple, but the architectural choices made early on cascade into radically different development paths. This answer charts two parallel projects – one using a traditional bespoke CMS backend, the other using an MCP-driven backend with a conversational AI agent as the primary interface – and identifies exactly where they diverge, what becomes redundant, and what gains importance.

---

## 1.2 Side-by-Side Development Timeline

| Phase | Project 1: Traditional CMS | Project 2: MCP-Driven Blog |
|---|---|---|
| **1. Requirements & Data Modeling** | Define post schema, user roles, categories, tags, media | Identical – same domain model regardless of interface |
| **2. Database Setup** | Choose DB, write migrations, set up ORM | Identical |
| **3. Core API / Business Logic** | CRUD routes for posts, categories, media, users | Same business logic, but exposed as MCP tool definitions instead of REST/GraphQL endpoints |
| **4. Authentication & Authorization** | Session management, login/logout, CSRF tokens, password hashing, role-based access | Token-based auth for MCP transport (JWT), but no login UI, no session cookies, no CSRF |
| **5. Admin UI** | Build admin panel: forms, validation, rich text editor, media uploader, preview | **Eliminated entirely** – the AI agent IS the interface |
| **6. Frontend (Public Site)** | Templating or SSG for public-facing blog | Identical |
| **7. Content Workflow** | Draft/publish state machine, scheduling, revision history via UI controls | Same state machine, but triggered conversationally; revision history via audit logs |
| **8. Testing** | Unit tests, integration tests, E2E tests for admin UI flows | Unit tests for tools, integration tests for MCP server, prompt regression tests |
| **9. Deployment** | Deploy app server + admin frontend + public site | Deploy MCP server + public site (no admin frontend to deploy) |
| **10. Maintenance** | Patch admin UI bugs, update dependencies for both frontend and backend | Maintain tool definitions, update guardrails, monitor agent behavior |

## 1.3 The Shared Foundation

Both projects share a surprisingly large common base. The first two phases are identical regardless of interface paradigm:

### 1.3.1 Data modeling

You still need a `posts` table with title, body, slug, status, timestamps, and author. You still need categories, tags, and media assets. The domain model does not care how humans (or agents) interact with it.

### 1.3.2 Database and migrations

Whether a human clicks "Publish" in a form or an AI agent calls a `publish_post` tool, the same row gets updated in the same database. Schema design, indexing strategy, and migration tooling are unchanged.

### 1.3.3 Core business logic

Validation rules like "a post must have a title" or "slugs must be unique" live in the business logic layer regardless. The difference is only in how that logic is invoked.

---

## 1.4 The Point of Divergence

The paths split at **Phase 3: how business logic is exposed to the outside world**.

In Project 1, you build HTTP endpoints:

```
# Traditional: REST endpoint
@app.route("/admin/posts", methods=["POST"])
@login_required
def create_post():
    form = PostForm(request.form)
    if form.validate():
        post = Post(title=form.title.data, body=form.body.data, ...)
        db.session.add(post)
        db.session.commit()
        return redirect(url_for("admin.posts"))
    return render_template("admin/post_form.html", form=form)
```

In Project 2, you define MCP tools:

```
# MCP-driven: tool definition
@mcp_server.tool()
async def create_post(
    title: str,
    body: str,
    status: Literal["draft", "published"] = "draft",
    tags: list[str] = [],
    category: str | None = None
) -> dict:
    """Create a new blog post.

    Args:
        title: The post title. Must be non-empty.
        body: The post content in markdown format.
        status: Whether to save as draft or publish immediately.
        tags: Optional list of tag names to apply.
        category: Optional category slug.
```

```
    """
    post = Post(title=title, body=body, status=status, ...)
    db.session.add(post)
    db.session.commit()
    return {"id": post.id, "slug": post.slug, "status": post.status}
```

The underlying `Post` creation is the same. But everything downstream of this divergence point – how the interface is built, tested, secured, and maintained – changes fundamentally.

---

## 1.5 What Becomes Redundant in the MCP-Driven Approach

### 1.5.1 Admin UI (the biggest elimination)

The traditional CMS admin panel is a major engineering effort:

- **Form components**: Text inputs, rich text editors (TinyMCE, ProseMirror), file uploaders, tag selectors, date pickers
- **Page layouts**: Dashboard, post list with filtering/sorting/pagination, edit screens, settings pages
- **Client-side state management**: Form dirty tracking, autosave, unsaved changes warnings
- **Responsive design**: Making the admin work on mobile
- **Accessibility**: ARIA labels, keyboard navigation, screen reader support for all admin controls

In the MCP-driven approach, all of this disappears. The AI agent's conversational interface replaces every form, button, and dropdown. As one practitioner put it, the entire workflow of "logging into admin interfaces, navigating menus and submenus, using clunky editors, previewing in separate windows, and publishing" is eliminated.

### 1.5.2 Server-Side Form Validation UI

You still validate data (a post needs a title), but you no longer need to render validation errors back into HTML forms with highlighted fields and inline messages. The MCP tool simply returns an error object, and the AI agent explains it conversationally.

### 1.5.3 Session Management and CSRF Protection

Traditional admin panels require: - Cookie-based sessions or JWT stored in httpOnly cookies - CSRF token generation and validation on every form submission - "Remember me" functionality - Session expiration and renewal logic

The MCP transport uses its own authentication model (typically short-lived JWTs with 15-minute lifespans via the MCP auth spec), and since there are no HTML forms, CSRF is not a concern.

### 1.5.4 Admin-Specific Routing and Middleware

No need for `/admin/*` route groups, admin layout templates, breadcrumb navigation, or admin-specific middleware chains. The MCP server exposes a flat set of tools – the "navigation" is handled by the agent's understanding of available capabilities.

### 1.5.5 Client-Side JavaScript for Admin

No webpack/vite configuration for the admin bundle. No React/Vue/Svelte components for the admin panel. No client-side form libraries. The entire admin frontend build pipeline is eliminated.

### 1.5.6 Password Reset and Account Management UI

No "forgot password" flow, no profile editing page, no avatar upload, no email verification screens.

---

## 1.6 What Increases in Importance

### 1.6.1 Tool Definitions and Descriptions

In a traditional CMS, a vague button label is a minor UX issue. In an MCP-driven system, a vague tool description means the AI agent literally cannot figure out when or how to use the tool. Tool descriptions become the primary "interface design" artifact:

```python
@mcp_server.tool()
async def update_post_status(
    post_id: int,
    new_status: Literal["draft", "published", "archived"]
) -> dict:
    """Change the publication status of an existing blog post.

    Use this to publish a draft, unpublish a live post, or archive
    old content. Publishing makes the post immediately visible on
    the public site. Archiving removes it from public view but
    preserves it in the database.

    Args:
        post_id: The numeric ID of the post to update.
        new_status: The target status.
    """
```

Every word in that docstring matters. It is the equivalent of designing the entire admin interaction flow for that feature.

### 1.6.2 Schema Design and Output Structure

With the introduction of MCP tool output schemas (released June 2025), the structure of what tools return becomes a design-time concern. The agent and client need to know the shape of responses ahead of time. This is analogous to API contract design but with the added constraint that the output must be interpretable by an LLM.

### 1.6.3 Safety Guardrails and Confirmation Workflows

A traditional CMS has the "Are you sure you want to delete this?" modal. In an MCP-driven system, you need equivalent safeguards but implemented differently:

- **Destructive action confirmation**: The MCP server can use elicitation (an MCP feature released June 2025) to request explicit confirmation before irreversible operations
- **Rate limiting on tool calls**: Preventing an agent from bulk-deleting content through a runaway loop
- **Scope restrictions**: Ensuring the agent can only modify posts, not drop database tables
- **Input sanitization**: The agent might pass unexpected or adversarial content; validation at the tool boundary is critical

### 1.6.4 Audit Logging

Traditional CMS systems often add audit logs as an afterthought. In an MCP-driven system, comprehensive logging is essential because the "user" is an AI agent whose behavior is less predictable than a human clicking through forms. Every tool invocation should be logged with full context: who initiated the request, what parameters were passed, what the outcome was, and the conversational context that led to the action.

### 1.6.5 Error Handling and Error Messages

Error messages in tool responses serve double duty: they must be machine-parseable (so the agent can retry or adjust) and human-readable (since the agent will relay them to the user). A raw stack trace is useless. A structured error with a clear message and suggested remediation is critical:

```
return {
    "error": "duplicate_slug",
    "message": "A post with the slug 'my-first-post' already exists.",
    "suggestion": "Try a different title or specify a custom slug."
}
```

### 1.6.6 Testing Strategy Shifts

Traditional CMS testing includes heavy E2E testing of admin UI flows (Cypress, Playwright). These disappear entirely but are replaced by:

- **Tool contract tests**: Does each tool accept the documented inputs and return the documented outputs?
- **Prompt regression tests**: Given a natural language request like “publish my draft about Python”, does the agent reliably call the right tool with the right parameters?
- **Adversarial input tests**: What happens when the agent passes malformed markdown, excessively long content, or attempts to access posts belonging to other users?
- **Integration tests against the MCP protocol**: Does the server correctly handle the MCP handshake, capability negotiation, and transport lifecycle?

### 1.6.7 Documentation as Interface

In a traditional CMS, you might write user documentation for the admin panel. In an MCP-driven system, the tool descriptions, parameter schemas, and example interactions ARE the documentation. They must be thorough enough for any MCP-compatible client to discover and use the tools correctly without external docs.

---

## 1.7 Development Effort Comparison

| Component | Traditional CMS | MCP-Driven | Delta |
|---|---|---|---|
| Data modeling | 1x | 1x | Same |
| Database setup | 1x | 1x | Same |
| Business logic | 1x | 1x | Same |
| API/Tool layer | 1x (REST routes) | 1x (MCP tools) | Similar effort, different shape |
| Admin UI | 3-5x (largest single cost) | 0x | **Eliminated** |
| Auth system | 2x | 0.5x (MCP auth only) | **Reduced** |
| Public frontend | 1x | 1x | Same |
| Tool descriptions / prompt engineering | 0x | 1.5x | **New cost** |
| Safety guardrails | 0.5x (basic modals) | 2x (comprehensive) | **Increased** |
| Audit logging | 0.5x (optional) | 1.5x (essential) | **Increased** |
| Testing | 2x (including E2E) | 1.5x (different shape) | Slightly reduced |
| **Total relative effort** | **~12-14x** | **~9-10x** | **~25-30% reduction** |

The net result is a meaningful reduction in total development effort, driven almost entirely by eliminating the admin UI. However, the savings are partially offset by new costs in guardrails, audit infrastructure, and tool design.

---

## 1.8 Tradeoffs and Practical Considerations

### 1.8.1 Where the MCP approach wins

- **Faster time to MVP**: Skipping the admin panel means you can have a functional content management system as soon as the MCP tools are defined and a compatible client (Claude, Cursor, etc.) is connected.
- **Lower maintenance burden**: No admin frontend to patch, update, or redesign. UI framework upgrades, CSS fixes, and browser compatibility issues vanish.
- **Flexible interface**: The same MCP server can be accessed from any MCP-compatible client – a CLI agent, a desktop app, a mobile assistant – without building separate UIs for each.
- **Natural language superpowers**: Complex multi-step operations ("create a new post summarizing this PDF, tag it with 'research', and schedule it for next Tuesday") become single conversational requests instead of multi-screen admin workflows.

### 1.8.2 Where the traditional approach wins

- **Visual content editing**: Rich text editing with real-time preview, drag-and-drop image placement, and WYSIWYG formatting are genuinely hard to replicate conversationally. Some content types need visual interfaces.
- **Discoverability**: A well-designed admin panel shows you what is possible. A conversational interface requires you to know (or guess) what to ask for. New users may struggle without visible affordances.
- **Determinism**: Clicking "Publish" always publishes. Saying "publish my post" might get misinterpreted if there are multiple drafts, or if the agent hallucinates a post ID. The probabilistic nature of LLM interpretation adds a class of errors that traditional UIs simply do not have.

- **Bulk operations with visual feedback**: Selecting 50 posts from a table view and batch-updating their category is trivial in a traditional UI. Describing the same operation conversationally requires careful specification and trust in the agent's execution.
- **Offline / low-connectivity**: Traditional admin panels work with just a web browser and a server. MCP-driven workflows depend on an AI client with an LLM backend, which may have latency, cost, or availability constraints.
- **Cost**: Every interaction with the MCP-driven blog incurs LLM inference costs. A traditional admin panel has near-zero marginal cost per interaction after deployment.

### 1.8.3 The hybrid reality

In practice, many production systems are converging on a hybrid model: a minimal traditional UI for operations requiring visual feedback (media management, layout preview, analytics dashboards) with MCP tools layered on top for content authoring and routine administration. Ghost CMS, Payload CMS, and others are already shipping MCP servers alongside their traditional admin panels, letting users choose the interface that fits the task.

---

## 1.9 Summary

The two projects share roughly 40% of their development work (data modeling, database, business logic, public frontend). They diverge at the interface layer, where the traditional project invests heavily in admin UI engineering while the MCP project invests in tool design, safety guardrails, and audit infrastructure. The MCP approach eliminates the single largest development cost – the admin panel – but introduces new categories of work that did not previously exist. The net effect is a leaner, faster development cycle with a different risk profile: fewer UI bugs, but new risks around agent reliability, LLM cost, and the inherent non-determinism of natural language interfaces.

---

*Sources:* - Plural: Replacing Admin Tools with AI Chat Interfaces via MCP

•

Ghost-MCP: Model Context Protocol Server for Ghost CMS

•

Why I'm Ditching Admin Panels for AI and MCP

•

MCP 2026 Roadmap

•

Conversational CMS: How MCP Transforms Content Creation

•

ElmapiCMS MCP Server

•

MCP's Biggest Growing Pains - The New Stack

*Question:*

Define the logical sequential order of backend development operations, accounting for the creation of a privileged MCP server for internal admin that replaces a traditional admin backend. Use a blog application as a concrete example, walking through each step from schema definition through to deployment.

# 2 Sequential Order of Backend Development Operations with an Admin MCP Server

This walkthrough defines the logical step-by-step order for building a modern backend, using a **blog application** as a concrete example. The key departure from traditional practice is that the internal admin interface – historically a web-based admin panel (think Django Admin, Rails Admin, Retool) – is replaced by a **privileged MCP server** that exposes administrative operations as tools consumable by an AI agent.

---

## 2.1 The Sequential Steps at a Glance

| Step | Operation | Output |
|---|---|---|
| 1 | Domain modelling and schema definition | ERD, migration files |
| 2 | Database provisioning and migration | Running database with tables |
| 3 | Shared data-access layer | ORM models / query modules |
| 4 | Public API creation | REST or GraphQL endpoints for end users |
| 5 | Admin MCP server creation | MCP tools for privileged internal operations |
| 6 | Security, auth, and audit wiring | JWT/OAuth on API; scoped auth + logging on MCP |
| 7 | Deployment and runtime configuration | Containers, secrets, transport config |

Each step depends on what came before it. The rationale for this ordering is discussed within each section.

---

## 2.2 Step 1: Domain Modelling and Schema Definition

Everything begins with the data model. Before you write a single line of server code, you need to know what you are storing and how entities relate.

### 2.2.1 Blog App Example

For a blog platform, the core entities are:

- **User** – authors and administrators
- **Post** – the blog entries themselves
- **Tag** – categorical labels for posts
- **Comment** – reader responses on posts

A schema definition using a migration tool like Prisma:

```
// schema.prisma

model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String
  role      Role     @default(AUTHOR)
  posts     Post[]
```

```
  createdAt DateTime @default(now())
}

enum Role {
  AUTHOR
  EDITOR
  ADMIN
}

model Post {
  id          Int       @id @default(autoincrement())
  title       String
  slug        String    @unique
  body        String
  status      PostStatus @default(DRAFT)
  author      User      @relation(fields: [authorId], references: [id])
  authorId    Int
  tags        Tag[]
  comments    Comment[]
  publishedAt DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

enum PostStatus {
  DRAFT
  PUBLISHED
  ARCHIVED
}

model Tag {
  id    Int    @id @default(autoincrement())
  name  String @unique
  posts Post[]
}

model Comment {
  id        Int      @id @default(autoincrement())
  body      String
  authorName String
  post      Post     @relation(fields: [postId], references: [id])
  postId    Int
  approved  Boolean  @default(false)
  createdAt DateTime @default(now())
}
```

### 2.2.2 Why This Comes First

The schema is the single source of truth. The public API, the admin MCP server, and all business logic derive from it. Changing the schema later forces changes everywhere downstream, so getting the domain model right (or at least directionally correct) is the highest-leverage early decision.

---

## 2.3 Step 2: Database Provisioning and Migration

With the schema defined, you create the actual database and apply the schema to it.

```
# Provision a PostgreSQL database (local dev example)
createdb blog_app
```

```
# Apply the Prisma schema as a migration
npx prisma migrate dev --name init
```

This step produces:

- A running PostgreSQL (or SQLite, MySQL, etc.) instance with all tables created
- A migration history that can be replayed in staging and production
- A generated Prisma Client (or equivalent ORM client) for typed database access

### 2.3.1 Why This Comes Second

You cannot write data-access code without a database to access. The migration step also validates that the schema is internally consistent (foreign keys resolve, unique constraints do not conflict, etc.).

---

## 2.4 Step 3: Shared Data-Access Layer

Before building either the public API or the admin MCP server, you create a shared layer of data-access functions. This is critical because the public API and the admin MCP server both need to read and write the same data, and duplicating query logic across them is a maintenance hazard.

```
// src/data/posts.ts
import { PrismaClient, PostStatus } from "@prisma/client";

const prisma = new PrismaClient();

export async function getPublishedPosts(page: number, pageSize: number) {
  return prisma.post.findMany({
    where: { status: PostStatus.PUBLISHED },
    include: { author: true, tags: true },
    orderBy: { publishedAt: "desc" },
    skip: (page - 1) * pageSize,
    take: pageSize,
  });
}

export async function createPost(data: {
  title: string;
  slug: string;
  body: string;
  authorId: number;
  tagIds?: number[];
}) {
  return prisma.post.create({
    data: {
      title: data.title,
      slug: data.slug,
      body: data.body,
      authorId: data.authorId,
      tags: data.tagIds
        ? { connect: data.tagIds.map((id) => ({ id })) }
        : undefined,
    },
  });
}

export async function bulkUpdatePostStatus(
  postIds: number[],
```

```
  status: PostStatus
) {
  return prisma.post.updateMany({
    where: { id: { in: postIds } },
    data: {
      status,
      publishedAt: status === PostStatus.PUBLISHED ? new Date() : undefined,
    },
  });
}

export async function moderateComments(
  commentIds: number[],
  approved: boolean
) {
  return prisma.comment.updateMany({
    where: { id: { in: commentIds } },
    data: { approved },
  });
}

export async function getAdminDashboardStats() {
  const [totalPosts, publishedPosts, pendingComments, totalUsers] =
    await Promise.all([
      prisma.post.count(),
      prisma.post.count({ where: { status: PostStatus.PUBLISHED } }),
      prisma.comment.count({ where: { approved: false } }),
      prisma.user.count(),
    ]);
  return { totalPosts, publishedPosts, pendingComments, totalUsers };
}
```

### 2.4.1 Why This Comes Third

Both the API (Step 4) and the MCP server (Step 5) import from this layer. Building it before either consumer prevents logic duplication and ensures consistent behavior. This is the same principle Plural applies in production: their MCP servers “reuse existing business logic and database connections rather than duplicating functionality in separate admin interfaces.”

---

## 2.5 Step 4: Public API Creation

The public API serves external consumers: the blog frontend, mobile apps, third-party integrations. It exposes only the operations and data that unauthenticated or reader-authenticated users should access.

```
// src/api/server.ts
import express from "express";
import { getPublishedPosts } from "../data/posts.js";
import { getPostBySlug, getPostComments } from "../data/posts.js";

const app = express();
app.use(express.json());

// Public: list published posts
app.get("/api/posts", async (req, res) => {
  const page = parseInt(req.query.page as string) || 1;
  const posts = await getPublishedPosts(page, 20);
```

```
  res.json({ posts, page });
});

// Public: get a single post by slug
app.get("/api/posts/:slug", async (req, res) => {
  const post = await getPostBySlug(req.params.slug);
  if (!post || post.status !== "PUBLISHED") {
    return res.status(404).json({ error: "Not found" });
  }
  res.json(post);
});

// Public: get approved comments for a post
app.get("/api/posts/:slug/comments", async (req, res) => {
  const comments = await getPostComments(req.params.slug, true);
  res.json(comments);
});

// Authenticated: submit a comment (rate-limited, validated)
app.post("/api/posts/:slug/comments", async (req, res) => {
  // Validation and rate limiting omitted for brevity
  const comment = await createComment({
    postSlug: req.params.slug,
    authorName: req.body.authorName,
    body: req.body.body,
  });
  res.status(201).json(comment);
});

app.listen(3000);
```

### 2.5.1 What the Public API Does NOT Include

The public API deliberately excludes:

- Bulk post status changes (publish, archive, unpublish)
- Comment moderation (approve/reject)
- User management (role changes, account creation)
- Dashboard statistics
- Direct database queries or schema inspection

These are admin operations. In a traditional stack, you would build a separate admin panel (a web UI with its own routes, controllers, and frontend) to handle them. In the AI-native approach, the admin MCP server handles them instead.

### 2.5.2 Why This Comes Fourth

The public API depends on the data-access layer (Step 3) and the database (Step 2). It does not depend on the admin MCP server, so it can be built and deployed independently.

---

## 2.6 Step 5: Admin MCP Server Creation

This is the step that replaces the traditional admin backend. Instead of building an admin web panel with its own UI, routes, and controllers, you define an MCP server that exposes administrative operations as **tools** and administrative data as **resources**.

An AI agent (Claude, or any MCP-compatible client) can then invoke these tools via natural language, subject to authorization.

```
// src/admin-mcp/server.ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";
import {
  bulkUpdatePostStatus,
  moderateComments,
  getAdminDashboardStats,
  createPost,
} from "../data/posts.js";
import { listUsers, updateUserRole } from "../data/users.js";

const server = new McpServer({
  name: "blog-admin",
  version: "1.0.0",
});

// --- RESOURCES (read-only data the agent can inspect) ---

server.resource("dashboard-stats", "admin://dashboard/stats", async (uri) => ({
  contents: [
    {
      uri: uri.href,
      mimeType: "application/json",
      text: JSON.stringify(await getAdminDashboardStats()),
    },
  ],
}));

server.resource("all-users", "admin://users", async (uri) => ({
  contents: [
    {
      uri: uri.href,
      mimeType: "application/json",
      text: JSON.stringify(await listUsers()),
    },
  ],
}));

// --- TOOLS (actions the agent can perform) ---

server.tool(
  "publish-posts",
  "Publish one or more draft posts by their IDs",
  { postIds: z.array(z.number()).min(1) },
  async ({ postIds }) => {
    const result = await bulkUpdatePostStatus(postIds, "PUBLISHED");
    return {
      content: [
        {
          type: "text",
          text: `Published ${result.count} post(s).`,
        },
      ],
    };
  }
);

server.tool(
```

```
  "archive-posts",
  "Archive one or more posts by their IDs",
  { postIds: z.array(z.number()).min(1) },
  async ({ postIds }) => {
    const result = await bulkUpdatePostStatus(postIds, "ARCHIVED");
    return {
      content: [
        {
          type: "text",
          text: `Archived ${result.count} post(s).`,
        },
      ],
    };
  }
);

server.tool(
  "moderate-comments",
  "Approve or reject comments by their IDs",
  {
    commentIds: z.array(z.number()).min(1),
    approved: z.boolean(),
  },
  async ({ commentIds, approved }) => {
    const result = await moderateComments(commentIds, approved);
    return {
      content: [
        {
          type: "text",
          text: `${approved ? "Approved" : "Rejected"} ${result.count} comment(s).`,
        },
      ],
    };
  }
);

server.tool(
  "change-user-role",
  "Change a user's role (AUTHOR, EDITOR, ADMIN)",
  {
    userId: z.number(),
    role: z.enum(["AUTHOR", "EDITOR", "ADMIN"]),
  },
  async ({ userId, role }) => {
    const user = await updateUserRole(userId, role);
    return {
      content: [
        {
          type: "text",
          text: `Updated ${user.name} to role ${role}.`,
        },
      ],
    };
  }
);

server.tool(
  "create-post",
  "Create a new blog post as a draft",
```

```
  {
    title: z.string(),
    slug: z.string(),
    body: z.string(),
    authorId: z.number(),
    tagIds: z.array(z.number()).optional(),
  },
  async (args) => {
    const post = await createPost(args);
    return {
      content: [
        {
          type: "text",
          text: `Created draft post "${post.title}" (ID: ${post.id}).`,
        },
      ],
    };
  }
);

// --- START ---
const transport = new StdioServerTransport();
await server.connect(transport);
```

### 2.6.1 MCP Client Configuration

The admin MCP server is registered in the AI client's configuration. For Claude Code, this would go in the project's `.mcp.json`:

```
{
  "mcpServers": {
    "blog-admin": {
      "command": "node",
      "args": ["src/admin-mcp/server.js"],
      "env": {
        "DATABASE_URL": "postgresql://admin:secret@localhost:5432/blog_app"
      }
    }
  }
}
```

### 2.6.2 What This Replaces

In a traditional stack, the admin panel for this blog would require:

- A separate frontend (React/Vue admin dashboard, or a framework-provided admin like Django Admin)
- Admin-specific API routes (`/admin/posts`, `/admin/users`, `/admin/comments`)
- Admin controllers and middleware
- Admin authentication flows (separate login page, session management)
- UI components for tables, forms, bulk actions, dashboards

The MCP server eliminates all of that. An authorized operator opens their AI client and says:

> "Show me the dashboard stats, then approve all pending comments on post 42, and publish posts 15, 16, and 17."

The agent reads the `dashboard-stats` resource, then calls `moderate-comments` and `publish-posts` in sequence. No admin UI was built, maintained, or deployed.

### 2.6.3 Why This Comes Fifth

The MCP server depends on the same data-access layer as the public API (Step 3). It is logically separate from the public API – different consumers, different authorization model, different transport. Building it after the public API ensures the shared data layer is already battle-tested. However, there is no hard dependency between Steps 4 and 5; they could be built in parallel since both depend only on Steps 1-3.

---

## 2.7 Step 6: Security, Authorization, and Audit Logging

Security is applied to both the public API and the admin MCP server, but with different models appropriate to each.

### 2.7.1 Public API Security

Standard web API security: JWT or session-based auth for authenticated routes, rate limiting, input validation, CORS.

### 2.7.2 Admin MCP Server Security

The MCP server requires a different, typically stricter, security model:

**Authentication:** JWT tokens with short lifespans (Plural uses ES256 JWTs with 15-minute expiry). The token contains the operator's identity and group memberships. The MCP server validates the token before executing any tool.

**Authorization:** Role-based or attribute-based access control on each tool. For example, only users with the `ADMIN` role can call `change-user-role`, while `EDITOR` can call `publish-posts` and `moderate-comments`.

```
// Middleware-style authorization check within a tool handler
server.tool(
  "change-user-role",
  "Change a user's role",
  { userId: z.number(), role: z.enum(["AUTHOR", "EDITOR", "ADMIN"]) },
  async (args, context) => {
    const caller = await validateJwt(context.meta?.authToken);
    if (caller.role !== "ADMIN") {
      return {
        content: [{ type: "text", text: "Denied: ADMIN role required." }],
        isError: true,
      };
    }
    // proceed with operation...
  }
);
```

**Audit logging:** Every tool invocation is logged with the caller's identity, the tool name, the arguments, the result, and a timestamp. This is non-negotiable for a privileged server.

**Human-in-the-loop for destructive operations:** For actions like deleting a user or purging all comments, the MCP server should require explicit confirmation. MCP clients like Claude Code already support this pattern by prompting the user before executing tools.

### 2.7.3 Why This Comes Sixth

Security is applied on top of working functionality. You need the API and MCP server to exist before you can wire authentication and authorization into them. In practice, you would design the security model earlier (during Step 1), but the implementation happens here.

---

## 2.8 Step 7: Deployment and Runtime Configuration

The final step is deploying the system. The public API and the admin MCP server have different deployment characteristics.

### 2.8.1 Public API

Deployed as a standard web service – containerized, behind a load balancer, with horizontal scaling. Nothing unusual here.

### 2.8.2 Admin MCP Server

The MCP server can be deployed in several ways depending on the transport:

- **Stdio transport (local):** The MCP server runs as a subprocess of the AI client. This is the simplest model and works well when the admin operator is a developer running Claude Code on their machine. The server has direct access to the database via the `DATABASE_URL` environment variable.
- **SSE or Streamable HTTP transport (remote):** The MCP server runs as a web service, accessible over HTTPS. This allows remote administration and is better suited for team environments. It requires the full authentication stack from Step 6.

```yaml
# docker-compose.yml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: blog_app
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data

  api:
    build: .
    command: node src/api/server.js
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgresql://postgres:secret@postgres:5432/blog_app

  admin-mcp:
    build: .
    command: node src/admin-mcp/server.js
    ports:
      - "3001:3001"
    environment:
      DATABASE_URL: postgresql://postgres:secret@postgres:5432/blog_app
      MCP_TRANSPORT: sse
      AUTH_JWT_SECRET: ${JWT_SECRET}

volumes:
  pgdata:
```

---

## 2.9 Summary: The Dependency Chain

```
Step 1: Schema Definition
   |
   v
Step 2: Database Provisioning + Migration
   |
   v
Step 3: Shared Data-Access Layer
   |
   +-----------+-----------+
   |                       |
   v                       v
Step 4: Public API    Step 5: Admin MCP Server
   |                       |
   +-----------+-----------+
               |
               v
      Step 6: Security + Audit
               |
               v
      Step 7: Deployment
```

Steps 4 and 5 are independent of each other and can be built in parallel. Everything else is strictly sequential. The critical insight is that Step 5 – the admin MCP server – occupies the same architectural position that a traditional admin panel would, but produces a fundamentally different artifact: a set of tool and resource definitions rather than a web UI. The admin's "interface" becomes whatever AI client the operator uses.

---

## 2.10 Why Replace the Admin Panel with an MCP Server?

The practical benefits are significant:

1. **No frontend to build or maintain.** Admin panels are notoriously tedious to build and quick to become stale. The MCP server is pure backend logic with tool descriptions.
2. **Natural language as the interface.** Instead of navigating a dashboard to find the comment moderation page, filtering by status, selecting comments, and clicking "Approve," the operator says: "Approve all pending comments from the last 24 hours." The agent figures out the filtering and calls the tool.
3. **Composability.** An agent can chain multiple admin operations in a single conversation: "Show me posts with fewer than 100 views that are still published, archive them, and send me a summary." This would require custom UI work in a traditional admin panel.
4. **Audit logging is natural.** Every tool invocation is a discrete, logged event with structured parameters. Traditional admin panels often have patchy audit trails because logging must be manually added to each controller action.
5. **Reduced attack surface.** The MCP server can run on a private network or as a local subprocess, never exposed to the public internet. A web-based admin panel needs to be web-accessible, which means it needs its own authentication, CSRF protection, and hardening.

The tradeoff is that this approach requires operators who are comfortable interacting with AI agents rather than clicking through a GUI. For engineering teams and technical operators, this is increasingly the default mode of work.

---

## 2.11 Sources

- Plural: Replacing Admin Tools with AI Chat Interfaces via MCP
- MCP Authorization: Securing MCP Servers with Fine-Grained Access Control (Cerbos)
- Understanding Authorization in MCP (Official Docs)
- MCP TypeScript SDK (GitHub)
- How to Build a To-Do List MCP Server Using TypeScript (freeCodeCamp)
- MCP Security Risks and Mitigations (Red Hat)

*Question:*

Consider a company developing a data-backed application with two MCPs: one for public access (unauthenticated) and another for privileged admin access (authenticated). Describe the use-case, both MCP servers, and the development and security approach.

# 3 Dual MCP Server Architecture: Public and Admin Access

## 3.1 The Scenario: CityTransit – A Public Transit Data Platform

Consider **CityTransit**, a company that aggregates public transportation data for a metropolitan area. They maintain a database of routes, schedules, real-time vehicle positions, service alerts, and fare information. CityTransit wants to expose this data through MCP servers so that AI assistants and agentic applications can query transit information on behalf of users.

They need two MCP servers:

1. **CityTransit Public MCP** – unauthenticated, read-only access to schedules, routes, and alerts. Any AI assistant can connect and answer questions like "When is the next bus on Route 42?" without credentials.
2. **CityTransit Admin MCP** – authenticated, privileged access for transit authority staff. Supports creating service alerts, modifying schedules, managing fare rules, and viewing ridership analytics. Only authorized personnel can invoke these tools.

This is a realistic pattern: the underlying data is the same, but the operations and access levels differ dramatically between public consumers and internal administrators.

---

## 3.2 The Two MCP Servers

### 3.2.1 Public MCP Server

The public server exposes **read-only tools and resources**. It has no authentication requirement, making it easy for any MCP client to connect and start querying transit data. The design principle is simple: if the data is already published on the agency's website, it can be served through this MCP.

**Capabilities:**

- Query routes and stops
- Look up schedules and next departures
- Read active service alerts
- Get fare information
- Access real-time vehicle positions

### 3.2.2 Admin MCP Server

The admin server exposes **read and write tools** behind OAuth 2.1 authentication. It is intended for internal staff using AI assistants to manage transit operations – updating schedules after a disruption, publishing service alerts, adjusting fares, and reviewing ridership data.

**Capabilities:**

- All public read capabilities (so admins do not need two connections)
- Create, update, and delete service alerts
- Modify route schedules
- Update fare rules
- View ridership analytics and reports
- Manage stop and route metadata

---

## 3.3 Tool Definitions

### 3.3.1 Public MCP Server Tools

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

const publicServer = new McpServer({
  name: "citytransit-public",
  version: "1.0.0",
});

publicServer.tool(
  "get_next_departures",
  "Get upcoming departures from a specific stop",
  {
    stop_id: z.string().describe("The stop identifier, e.g. 'STOP-1042'"),
    route_id: z.string().optional().describe("Filter by route"),
    limit: z.number().min(1).max(20).default(5).describe("Number of results"),
  },
  async ({ stop_id, route_id, limit }) => {
    const departures = await db.departures.findUpcoming({ stop_id, route_id, limit });
    return {
      content: [{ type: "text", text: JSON.stringify(departures, null, 2) }],
    };
  }
);

publicServer.tool(
  "search_routes",
  "Search for transit routes by name, number, or area",
  {
    query: z.string().describe("Search term for route name or number"),
    transport_type: z.enum(["bus", "rail", "tram", "all"]).default("all"),
  },
  async ({ query, transport_type }) => {
    const routes = await db.routes.search({ query, transport_type });
    return {
      content: [{ type: "text", text: JSON.stringify(routes, null, 2) }],
    };
  }
);

publicServer.tool(
  "get_active_alerts",
  "Get current service alerts and disruptions",
  {
    route_id: z.string().optional().describe("Filter alerts by route"),
    severity: z.enum(["info", "warning", "critical", "all"]).default("all"),
  },
  async ({ route_id, severity }) => {
    const alerts = await db.alerts.findActive({ route_id, severity });
    return {
      content: [{ type: "text", text: JSON.stringify(alerts, null, 2) }],
    };
  }
);

publicServer.tool(
  "get_fare_info",
```

```
  "Look up fare pricing for a journey",
  {
    origin_stop: z.string().describe("Origin stop ID"),
    destination_stop: z.string().describe("Destination stop ID"),
    passenger_type: z.enum(["adult", "child", "senior", "student"]).default("adult"),
  },
  async ({ origin_stop, destination_stop, passenger_type }) => {
    const fare = await db.fares.calculate({ origin_stop, destination_stop,
passenger_type });
    return {
      content: [{ type: "text", text: JSON.stringify(fare, null, 2) }],
    };
  }
);
```

### 3.3.2 Admin MCP Server Tools

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

const adminServer = new McpServer({
  name: "citytransit-admin",
  version: "1.0.0",
});

// Re-export all public tools (imported from shared module)
registerPublicTools(adminServer);

// --- Write operations (admin only) ---

adminServer.tool(
  "create_service_alert",
  "Publish a new service alert for riders",
  {
    title: z.string().min(5).max(200).describe("Alert headline"),
    description: z.string().min(10).max(2000).describe("Full alert text"),
    severity: z.enum(["info", "warning", "critical"]),
    affected_routes: z.array(z.string()).min(1).describe("Route IDs affected"),
    start_time: z.string().datetime().describe("Alert start time (ISO 8601)"),
    end_time: z.string().datetime().optional().describe("Alert end time, if known"),
  },
  async ({ title, description, severity, affected_routes, start_time, end_time }, context)
=> {
    const user = context.authInfo.user; // from validated OAuth token
    await auditLog.record("create_alert", user, { title, severity });

    const alert = await db.alerts.create({
      title, description, severity, affected_routes,
      start_time, end_time,
      created_by: user.id,
    });
    return {
      content: [{ type: "text", text: `Alert created: ${alert.id} -- "${title}"` }],
    };
  }
);

adminServer.tool(
```

```
  "update_schedule",
  "Modify departure times for a route on a specific date",
  {
    route_id: z.string(),
    effective_date: z.string().date().describe("Date the change applies (YYYY-MM-DD)"),
    modifications: z.array(z.object({
      stop_id: z.string(),
      old_time: z.string().describe("Current departure time (HH:MM)"),
      new_time: z.string().describe("Updated departure time (HH:MM)"),
    })),
    reason: z.string().min(5).describe("Reason for modification"),
  },
  async ({ route_id, effective_date, modifications, reason }, context) => {
    const user = context.authInfo.user;
    if (!user.roles.includes("schedule_manager")) {
      throw new Error("Insufficient permissions: schedule_manager role required");
    }

    await auditLog.record("update_schedule", user, { route_id, effective_date, reason });
    const result = await db.schedules.batchUpdate({ route_id, effective_date,
modifications });
    return {
      content: [{ type: "text", text: `Updated ${result.modifiedCount} departures on route
${route_id}` }],
    };
  }
);

adminServer.tool(
  "get_ridership_analytics",
  "Retrieve ridership statistics for a route or time period",
  {
    route_id: z.string().optional(),
    start_date: z.string().date(),
    end_date: z.string().date(),
    granularity: z.enum(["hourly", "daily", "weekly"]).default("daily"),
  },
  async ({ route_id, start_date, end_date, granularity }, context) => {
    const user = context.authInfo.user;
    if (!user.roles.includes("analytics_viewer")) {
      throw new Error("Insufficient permissions: analytics_viewer role required");
    }

    const stats = await db.analytics.query({ route_id, start_date, end_date,
granularity });
    return {
      content: [{ type: "text", text: JSON.stringify(stats, null, 2) }],
    };
  }
);
```

---

## 3.4 Development Approach

### 3.4.1 Shared Schema and Data Layer

Both servers read from the same database. The team should build a **shared data access layer** as an internal package:

```
packages/
  citytransit-data/        # Shared: DB models, queries, validation
    src/
      models/              # Route, Stop, Alert, Schedule, Fare
      queries/             # Parameterized queries with row-level filtering
      validation/          # Zod schemas reused across both servers
servers/
  public-mcp/             # Public server: imports from citytransit-data
  admin-mcp/              # Admin server: imports from citytransit-data
```

This monorepo structure ensures that both servers share the same data models and query logic. Schema changes propagate to both servers through the shared package, preventing drift.

### 3.4.2 Separate Tool Registrations

Tools are defined in a shared module where possible, then selectively registered on each server:

- **Public tools** are defined once and registered on both servers. The public server imports them directly; the admin server re-registers them so that admins have full read access without needing a second connection.
- **Admin tools** are defined only in the admin server package. They are never importable by the public server – enforced at the package level, not just at runtime.

This separation means that even a misconfigured public server cannot accidentally expose admin tools because the code for those tools does not exist in its dependency tree.

### 3.4.3 Database Access Patterns

The shared data layer should enforce read-only access for public queries at the connection level:

```typescript
// citytransit-data/src/connections.ts
export const readPool = new Pool({
  connectionString: process.env.DATABASE_URL_READONLY,
  // Uses a PostgreSQL role with SELECT-only grants
});

export const writePool = new Pool({
  connectionString: process.env.DATABASE_URL_READWRITE,
  // Uses a PostgreSQL role with SELECT, INSERT, UPDATE, DELETE grants
});
```

The public server only has access to `readPool` in its environment. The admin server has both. This is defense in depth: even if a bug in a public tool somehow tried to execute a write query, the database connection would reject it.

---

## 3.5 Security Considerations

### 3.5.1 Authentication

The public server requires no authentication. The admin server uses **OAuth 2.1** following the MCP specification's guidance that MCP servers act as OAuth 2.0 Resource Servers:

- The admin server publishes a `/.well-known/oauth-protected-resource` metadata document, identifying the authorization server and required scopes.
- MCP clients obtain tokens from the organization's identity provider (e.g., Okta, Auth0, or an internal IdP).

- The admin server validates access tokens on every request, checking signature, expiry, audience, and the `resource` indicator per RFC 8707.
- Tokens are short-lived (15-minute access tokens with refresh tokens for longer sessions).

### 3.5.2 Authorization (Role-Based)

Authentication tells you *who* is calling; authorization tells you *what they can do*. The admin server enforces role-based access control internally:

| Role | Permissions |
|---|---|
| `alert_publisher` | Create and update service alerts |
| `schedule_manager` | Modify route schedules |
| `fare_admin` | Update fare rules |
| `analytics_viewer` | Access ridership reports |
| `super_admin` | All of the above |

Roles are encoded as claims in the OAuth token or looked up from an internal directory. Each admin tool checks the caller's roles before executing.

### 3.5.3 Rate Limiting

Even though the public server has no authentication, it must protect itself from abuse:

- **Per-IP rate limiting** on the public server (e.g., 100 requests per minute per IP).
- **Per-token rate limiting** on the admin server (e.g., 500 requests per minute per token).
- Rate limit headers returned on every response so clients can self-throttle.
- Graduated backoff: soft limits return 429 with a `Retry-After` header; hard limits (10x the soft limit) result in temporary IP/token bans.

### 3.5.4 Input Validation

Both servers use **Zod schemas** (as shown in the tool definitions) to validate all inputs before they reach the data layer. This is the first line of defense against injection attacks and malformed data.

Additional validation at the data layer:

- Parameterized queries only – no string interpolation in SQL.
- Date range queries capped to prevent resource exhaustion (e.g., max 90-day window for analytics).
- String length limits on all text fields.
- Enumerated values enforced at the schema level (severity, transport type, passenger type).

### 3.5.5 Audit Logging

Every write operation on the admin server is recorded in an append-only audit log:

```
interface AuditEntry {
  timestamp: string;      // ISO 8601
  user_id: string;        // From OAuth token
  action: string;         // Tool name
  parameters: object;     // Sanitized input (no secrets)
  result: "success" | "failure";
  ip_address: string;
```

```
  token_jti: string;      // JWT ID for traceability
}
```

Audit logs are stored separately from the application database and are immutable. They enable post-incident investigation and compliance reporting.

### 3.5.6 Transport Security

- Both servers communicate over HTTPS only. The MCP SDK automatically upgrades HTTP to HTTPS.
- The admin server additionally supports mutual TLS (mTLS) for machine-to-machine integrations where an AI agent operates on behalf of a service account rather than a human.

---

## 3.6 Deployment and Operational Concerns

### 3.6.1 Separate Deployments

The two servers should be deployed as **independent services** with separate scaling, monitoring, and failure domains:

```
                    +------------------+
Public clients -->| Load Balancer    |
                    | (no auth check)  |
                    +--------+---------+
                             |
                    +--------v---------+
                    | citytransit-     |
                    | public-mcp       |
                    | (read-only DB)   |
                    +--------+---------+
                             |
                    +--------v---------+
                    |   PostgreSQL     |
                    |   (read replica) |
                    +------------------+

                    +------------------+
Admin clients --->| Load Balancer    |
                    | (OAuth token     |
                    |  pre-validation) |
                    +--------+---------+
                             |
                    +--------v---------+
                    | citytransit-     |
                    | admin-mcp        |
                    | (read/write DB)  |
                    +--------+---------+
                             |
                    +--------v---------+
                    |   PostgreSQL     |
                    |   (primary)      |
                    +------------------+
```

Key deployment decisions:

- **Public server** connects to a **read replica** of the database, providing natural isolation. Heavy public query traffic cannot degrade write performance for admins.

- **Admin server** connects to the **primary** database for write operations and can also read from it directly (or from a replica for analytics queries that tolerate slight lag).
- Each server has its own container image, scaling policy, and health checks.
- The public server can scale horizontally without limit since it is stateless and read-only. The admin server scales more conservatively because write operations must be serialized at the database level.

### 3.6.2 Environment Isolation

```
Environment     Public MCP              Admin MCP
-----------     ----------              ---------
Production      public.mcp.citytransit  admin.mcp.citytransit
Staging         public.stg.citytransit  admin.stg.citytransit
Development     localhost:3100          localhost:3200
```

Staging environments use a separate database snapshot, refreshed nightly. Developers run both servers locally against a Docker-composed PostgreSQL instance with seed data.

### 3.6.3 Monitoring and Alerting

Both servers emit structured logs and metrics:

- **Public server metrics**: request rate, latency percentiles (p50/p95/p99), error rate, cache hit ratio, rate-limit rejections per minute.
- **Admin server metrics**: all of the above plus write operation counts by type, authentication failures, authorization denials, audit log volume.
- Alert on: error rate exceeding 1% over 5 minutes, p99 latency exceeding 2 seconds, any authorization denial spike (possible credential compromise), audit log write failures.

### 3.6.4 Versioning and Backwards Compatibility

Both servers follow semantic versioning. The `name` and `version` fields in the MCP server metadata allow clients to detect capabilities:

- Public server versions increment independently of the admin server.
- When a breaking change is needed (removing a tool, changing a parameter), the old version runs in parallel for a deprecation period.
- The shared data package version is pinned in both servers' dependencies, and updates are coordinated.

### 3.6.5 Secret Management

Following MCP security best practices, credentials are never stored in code or configuration files:

- Database connection strings are injected via environment variables from a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault).
- OAuth signing keys are rotated automatically.
- The public server has minimal secrets (just the read-only database URL). The admin server has more (read-write database URL, audit log credentials, IdP configuration), so its deployment pipeline has stricter access controls.

---

## 3.7 Summary

The dual MCP server pattern – one public, one admin – is a natural fit for data-backed applications where the same underlying data needs both broad read access and controlled write access. The key architectural principles are:

1. **Share the data layer, separate the tool surfaces.** Both servers use the same models and queries, but admin tools exist only in the admin server's codebase.
2. **Enforce access boundaries at multiple levels.** Database connection roles, OAuth token validation, and role-based tool-level checks all reinforce each other.
3. **Deploy independently.** Separate scaling, separate failure domains, separate database connections (read replica vs. primary).
4. **Audit everything on the write path.** Every admin action is logged with the actor, action, parameters, and outcome.
5. **Validate all inputs regardless of trust level.** Zod schemas on every tool, parameterized queries in the data layer, and rate limiting on both servers.

This approach lets the transit authority confidently expose their data to the broader AI ecosystem through the public MCP while maintaining tight control over operational changes through the admin MCP.

---

*Sources:* - MCP Authorization Specification

•

MCP Authentication and Authorization Guide (Stytch)

•

MCP Spec Updates from June 2025 (Auth0)

•

OWASP Practical Guide for Secure MCP Server Development

•

MCP Authorization with Fine-Grained Access Control (Cerbos)

•

Authorization for MCP: OAuth 2.1 and Best Practices (Oso)

•

MCP Server Best Practices for 2026 (CData)

•

TypeScript MCP SDK (GitHub)

•

Best Practices for Securing Credentials in MCP Servers

*Question:*

What tooling exists today for unifying API and MCP tool development, so we can visualize exactly which routes are exposed as API endpoints, which are exposed as MCP tools, and which are both?

# 4 Tooling for Unified API and MCP Tool Development with Visibility

## 4.1 The Problem

Today's common pattern is to build a REST API first, then bolt on an MCP server as a separate layer that wraps those endpoints. This creates a maintenance burden: two codebases to keep in sync, no single view of what is exposed where, and easy drift between the API surface and the tool surface. The ideal is a single development workflow where you define your business logic once and declaratively control whether each operation is exposed as an API endpoint, an MCP tool, or both – with a dashboard or manifest that makes this mapping visible at a glance.

No single tool fully solves this end-to-end today, but a strong ecosystem is rapidly forming. The approaches fall into three tiers: framework-level libraries, code-generation platforms, and gateway/registry infrastructure.

---

## 4.2 Tier 1: Framework-Level Libraries (Define Once, Expose Both)

These let you write your backend logic once and serve it simultaneously as REST and MCP from the same process.

### 4.2.1 FastAPI-MCP (tadata-org)

FastAPI-MCP is the most direct answer to the unification question for Python shops. You mount an MCP server directly onto your FastAPI application with zero configuration, and every route automatically becomes an MCP tool. Crucially, it supports selective exposure through four filtering mechanisms:

- `include_operations` / `exclude_operations` – whitelist or blacklist by operation ID
- `include_tags` / `exclude_tags` – filter by OpenAPI tag

This means you can tag a route as `"api-only"` or `"mcp-only"` and get precise control. The operation ID from your FastAPI route becomes the MCP tool name, and request/response schemas are preserved from OpenAPI. Authentication flows through your existing FastAPI dependency injection, so there is no separate auth layer for MCP.

```python
from fastapi import FastAPI
from fastapi_mcp import FastApiMCP

app = FastAPI()

# Your normal routes
@app.get("/users/{user_id}", tags=["api-and-mcp"])
async def get_user(user_id: int): ...

@app.post("/internal/migrate", tags=["api-only"])
async def migrate_db(): ...

# Mount MCP -- expose only routes tagged for MCP
mcp = FastApiMCP(app, exclude_tags=["api-only"])
mcp.mount()
```

**Visibility**: You can inspect `mcp.tools` at runtime or write a startup hook that prints a table of which routes are API-only, MCP-only, or both. There is no built-in visual dashboard, but the tag-based system makes it straightforward to build one or generate a static manifest.

#### 4.2.2 FastMCP (Prefect/jlowin)

FastMCP takes the opposite entry point: you start from an MCP server and can import existing OpenAPI specs or FastAPI apps into it. Its `route_map_fn` callback gives you per-route control over whether an endpoint becomes a TOOL, RESOURCE, RESOURCE_TEMPLATE, or is EXCLUDED entirely.

```
from fastmcp import FastMCP

mcp = FastMCP.from_openapi(
    url="https://api.example.com/openapi.json",
    route_map_fn=lambda route: "TOOL" if route.method == "POST" else "RESOURCE"
)
```

FastMCP 3.0 added server composition, allowing you to merge multiple MCP servers (or API specs) into a single unified server. This is useful for organizations with many microservices that want a single MCP surface.

**Visibility**: The `route_map_fn` pattern is inherently explicit – you can log every routing decision and produce a mapping table at startup.

---

### 4.3 Tier 2: Code Generation Platforms (OpenAPI as Single Source of Truth)

These tools take your OpenAPI specification and generate a fully functional MCP server alongside (or instead of) traditional SDKs. The OpenAPI spec becomes the canonical definition, and both REST and MCP surfaces are derived from it.

#### 4.3.1 Speakeasy

Speakeasy generates production-ready, self-hostable MCP server code (TypeScript) from your OpenAPI spec. The generated server is real code you own and can customize, not a hosted proxy. Notable features:

- Generates one tool per API endpoint by default, or a dynamic mode with meta-tools (`list_api_endpoints`, `get_api_endpoint_schema`, `invoke_api_endpoint`) that let the LLM discover endpoints at runtime
- Full OAuth 2.1 support, API keys, bearer tokens
- JQ-based response transforms to shape data for optimal LLM consumption
- Deployment to Cloudflare Workers, Docker, or any infrastructure

**Visibility**: Because the OpenAPI spec is the source of truth, you always know exactly which endpoints exist. The generated code makes the mapping explicit – each tool maps to a named operation.

#### 4.3.2 Stainless

Stainless generates MCP servers alongside SDKs from OpenAPI specs, for free. Their architecture is distinctive: instead of one tool per endpoint, they generate a code execution tool and a docs search tool. The LLM writes SDK code that the MCP server executes, which is more token-efficient for large APIs.

They also support a `--tools=dynamic` mode that exposes three meta-tools for endpoint discovery and invocation, and handle client-specific quirks (Cursor's 40-tool limit, OpenAI's lack of root `anyOf` support).

### 4.3.3 AWS OpenAPI MCP Server

AWS's OpenAPI MCP Server dynamically creates MCP tools from OpenAPI specifications at runtime. No code generation step – you point it at a spec and it creates tools on the fly.

### 4.3.4 openapi-mcp-generator

openapi-mcp-generator is a CLI tool that generates MCP servers that proxy to existing REST APIs. Simpler than Speakeasy or Stainless, useful for quick prototyping.

---

## 4.4 Tier 3: Gateway and Registry Infrastructure (Centralized Visibility)

This is where the "visualization" part of the question gets the most direct answer. Gateways sit in front of your APIs and MCP servers, providing a unified registry, dashboard, and governance layer.

### 4.4.1 IBM ContextForge

ContextForge is the most feature-complete open-source option for centralized visibility. It is a gateway, registry, and proxy that federates MCP servers, A2A servers, and REST/gRPC APIs behind a single endpoint. Key capabilities:

- **Admin Dashboard** ( `/admin` ) with a web UI for managing servers and tools, including search, filtering, and pagination
- **Tool Registry** with bulk import (up to 200 tools per request) and per-tool metadata
- **MCP Inspector** for browsing tools, prompts, and resources in real time and invoking tools with JSON params
- **Protocol conversion** between stdio, SSE, and Streamable HTTP
- Converts REST API endpoints to MCP directly within the gateway

This is the closest thing to the "see which routes are API endpoints, which are MCP tools, and which are both" dashboard described in the question.

### 4.4.2 Azure API Management

Azure API Management offers one-click conversion of managed REST APIs into MCP servers, plus the ability to proxy existing MCP servers behind the same API gateway. Combined with Azure API Center, you get a unified catalog of traditional APIs, AI model APIs, MCP servers, and agent APIs – all in one registry with governance controls.

This is the enterprise-grade answer: if you already manage your APIs through Azure APIM, you can expose any of them as MCP tools without writing code, and the portal gives you full visibility into what is exposed where.

### 4.4.3 Microsoft MCP Gateway (Kubernetes)

Microsoft's MCP Gateway is a reverse proxy and management layer for MCP servers in Kubernetes environments, handling session-aware routing and lifecycle management. More infrastructure-focused than ContextForge, but useful for production deployments at scale.

### 4.4.4 Portkey, MintMCP, and Other Commercial Gateways

Commercial MCP gateways like Portkey and MintMCP provide managed dashboards with authentication, rate limiting, observability (Prometheus metrics, OpenTelemetry tracing), and tool registries. These are "unified LLM and tool gateways" that show all your MCP tools and API endpoints from a single pane of glass.

---

## 4.5 Emerging Patterns

### 4.5.1 The "OpenAPI-as-Contract" Pattern

The strongest emerging pattern is using OpenAPI as the single source of truth for both REST and MCP surfaces. You write your OpenAPI spec (or generate it from framework annotations), then derive both the REST server and the MCP server from it. Tools like Speakeasy, Stainless, FastMCP, and AWS's MCP server all support this. The benefit is that your API and MCP tool surfaces can never drift – they are both generated from the same document.

### 4.5.2 Decorator-Based Dual Exposure

FastAPI-MCP and FastMCP demonstrate a pattern where decorators or tags on your route handlers control exposure. This is the most developer-friendly approach for greenfield projects:

```python
@app.post("/send-email", tags=["mcp-tool", "api"])
async def send_email(to: str, subject: str, body: str): ...

@app.get("/health", tags=["api-only"])
async def health_check(): ...
```

The tags serve as the "visualization" – you can grep your codebase or generate a report.

### 4.5.3 Gateway-Level Unification

For organizations with existing APIs that cannot be refactored, gateways like ContextForge and Azure APIM provide unification at the infrastructure layer. You register your APIs and MCP servers in the gateway, and its admin UI becomes the single source of truth for what is exposed where.

### 4.5.4 Dynamic Tool Discovery

Stainless and Speakeasy's "dynamic" modes (meta-tools that let the LLM discover and invoke endpoints at runtime) represent a shift away from static tool registration. This avoids the tool-count explosion problem and works well for large APIs, but makes the "visibility" question more nuanced since the available tools depend on runtime queries.

---

## 4.6 Practical Recommendations

1. **For new Python projects**: Start with **FastAPI-MCP**. Define your routes with tags that explicitly mark exposure mode (`api-only`, `mcp-only`, `both`). Write a startup hook or CLI command that prints a table of all routes with their exposure status.
2. **For existing APIs with OpenAPI specs**: Use **Speakeasy** or **FastMCP's OpenAPI integration** to generate MCP servers from your spec. The spec itself becomes your visibility tool – annotate it with `x-mcp-expose: true/false` extension fields if you need fine-grained control.
3. **For enterprise environments with many services**: Deploy **IBM ContextForge** or **Azure API Management** as a gateway. Register all your APIs and MCP servers in it and use the admin dashboard for visibility.
4. **For a custom visualization dashboard**: None of the current tools provide a purpose-built "API vs MCP mapping" UI. Build a lightweight one by:
   - Parsing your OpenAPI spec for all endpoints
   - Querying your MCP server's `tools/list` endpoint
   - Cross-referencing the two lists to produce a matrix of endpoint, API-exposed, MCP-exposed, auth-required

5. **For future-proofing**: Adopt the OpenAPI-as-contract pattern regardless of which framework you choose. This keeps your options open as the tooling matures and ensures that API and MCP surfaces stay in sync by construction.

---

## 4.7 Summary Table

| Tool | Type | Language | Selective Exposure | Visual Dashboard | Open Source |
|---|---|---|---|---|---|
| FastAPI-MCP | Framework library | Python | Yes (tags, operation IDs) | No (programmatic) | Yes |
| FastMCP | Framework library | Python | Yes (route_map_fn) | No (programmatic) | Yes |
| Speakeasy | Code generator | TypeScript | Yes (OpenAPI filtering) | No | No (commercial) |
| Stainless | Code generator | TypeScript | Yes (dynamic tools) | No | Free tier |
| IBM ContextForge | Gateway/Registry | Python | Yes (per-tool config) | Yes (Admin UI) | Yes |
| Azure API Management | Gateway/Registry | N/A (managed) | Yes (portal config) | Yes (Azure Portal) | No (cloud service) |
| AWS OpenAPI MCP | Runtime generator | TypeScript | Partial | No | Yes |

The tooling gap that remains is a unified development experience where you annotate your business logic once with exposure metadata and get a live, interactive dashboard showing the full API/MCP mapping. FastAPI-MCP with ContextForge comes closest to this today, but a truly integrated solution – a framework with a built-in dev dashboard showing the dual surface – has not yet emerged.